

# Comparative Analysis of Deep Learning Architectures for Gesture Recognition

**Authors:** Mert Korkmaz (S034587), Emre Kolbakır (S034548), Seden Mutaf (S034353)

## 1. Introduction

Touchless human-computer interaction has become increasingly important in smart environments where convenience, hygiene, and accessibility matter. In smart homes, users typically interact with devices through touch or voice, but these methods have limitations such as noise sensitivity, privacy concerns, or physical constraints. Gesture-based interaction offers a natural alternative by enabling control through simple hand movements.

From a machine learning perspective, gesture recognition is a spatiotemporal problem, as gestures depend not only on hand appearance but also on motion over time. For example, gestures like swipe left and swipe right may look similar in single frames but differ when motion is considered.

In this project, we aim to build a gesture-command recognition system for touchless smart home control. Each input will be a short video clip containing a single gesture, and the output will be a corresponding command class. We formulate this as a controlled video classification problem and compare multiple deep learning architectures to analyze the trade-off between accuracy and computational efficiency.

## 2. Problem Statement

The goal of this project is to classify short gesture video clips into predefined smart home control commands such as swipe left, swipe right, thumb up, thumb down, and similar gestures.

We will use a subset of the Jester dataset, selecting around 8–10 distinct and relevant gesture classes. The dataset will be split into balanced training, validation, and test sets to ensure fair comparisons.

This task is challenging because:

- Some gestures differ mainly in motion rather than static appearance
- Variations in performer style, speed, and orientation increase complexity
- High-accuracy models may also be computationally expensive

Therefore, our objective is not only to achieve high accuracy but also to compare different modeling approaches in terms of both performance and efficiency.

We will evaluate models using:

- Top-1 accuracy
- Macro F1-score
- Confusion matrix
- Inference latency and model size

We expect 3D CNN models to achieve the highest accuracy but at a higher computational cost, while landmark-based models will be more efficient but potentially less robust. CNN-LSTM models are expected to provide a balance between these two extremes.

## 3. Technical Approach

We plan to compare three different deep learning approaches for gesture recognition. All models will be trained and evaluated on the same selected subset of gesture classes so that the results are directly comparable.

### 3.1 Data Preparation

For this project, we utilize a curated subset of the 20BN-JESTER dataset, which is provided in a pre-extracted frame format where each gesture is contained in a dedicated folder of 37 RGB images. Since the temporal segmentation is already defined by the dataset structure, our preprocessing focuses on standardizing these sequences for model ingestion: each frame is resized to a common 224 x 224 spatial resolution, and pixel values are normalized to a [0, 1] range to ensure numerical stability during training. For the landmark-based pipeline, we pass this fixed sequence of frames through the MediaPipe Hands API to extract 21 3D keypoints per frame, resulting in a coordinate tensor (37 x 63), while the RGB-based pipelines (CNN-LSTM and 3D-CNN) treat the folder contents as a single spatiotemporal volume. To handle slight variations in the number of frames across folders, we implement a simple padding or truncation strategy to ensure a uniform input shape for all neural network layers.

### 3.2 Landmark-Based Sequence Model

Our first method will use hand landmarks rather than raw pixel values. We plan to use MediaPipe Hands to extract 3D hand landmark coordinates from each sampled frame. Since each frame can be represented as a small set of hand keypoints, this approach greatly reduces input dimensionality and focuses only on geometric motion information.

The resulting landmark sequence will be fed into a temporal sequence model such as a BiLSTM or GRU, followed by a fully connected classification layer. The main advantage of this method is efficiency: it is lightweight, memory-friendly, and suitable for real-time inference. However, it may be more sensitive to landmark extraction failures and may lose useful visual cues such as hand appearance, orientation, or background contrast that could help distinguish ambiguous gestures.

### 3.3 CNN-LSTM Model

Our second method will be a frame-based hybrid architecture. In this approach, each sampled frame will first be processed by a pretrained 2D convolutional neural network such as **MobileNetV2** or **ResNet18** to extract spatial features. These frame-level embeddings will then be passed into an **LSTM** that models temporal dependencies across the video sequence. The final temporal representation will be used to predict the gesture class.

This method is attractive because it combines the strength of pretrained image feature extraction with explicit temporal modeling. Compared with the landmark-based approach, it can exploit richer visual information. Compared with a full 3D CNN, it is usually easier to train and less computationally expensive. We expect this model to provide a strong baseline and possibly the best trade-off between performance and efficiency.

### 3.4 3D Convolutional Video Model

Our third method will use a native video architecture based on **3D convolutions**, such as **R3D-18**. Unlike the CNN-LSTM model, which processes space and time separately, a 3D CNN learns spatiotemporal patterns jointly by convolving across both spatial and temporal dimensions at the same time.

The motivation for this approach is that gestures are inherently dynamic and may be better represented by a model that directly observes motion evolution within the clip. We expect this method to achieve the highest raw classification accuracy, especially for gestures that are difficult to distinguish using only frame-level appearance. However, this model is likely to require more GPU memory, longer training time, and slower inference than the other two approaches.

### 3.5 Training and Comparison Plan

All models will be trained using the same dataset split and optimized for multi-class classification using cross-entropy loss. Hyperparameters such as learning rate, batch size, and sequence length will be selected based on validation performance. We will compare the models using the evaluation criteria described above: Top-1 accuracy, macro F1-score, confusion matrix, inference latency, and model size.